

## Status Explanation:

As shown in the snippet, the integration between the operational backend (Node.js) and the analytical backend (ML Service) is established using internal HTTP calls (axios). This confirms the microservices architecture proposed in Chapter 4 is functional.

### 8.3.2 Machine Learning Inference Engine (Python)

The predictive capabilities of SHIFTY are powered by Python scripts running in a containerized environment. We have completed the inference script that loads the pre-trained XGBoost model and serves predictions to the backend.

## Implementation Logic:

The system loads a serialized .pkl file (the trained model) upon startup. When a request is received, the input features (distance, fragility, volume) are converted into a NumPy array and passed to the model.

### Code Snippet 2: Price Prediction Service (app.py)

```
from flask import Flask, request, jsonify
import joblib
import numpy as np
import pandas as pd

app = Flask(__name__)

# Load the trained XGBoost model and Random Forest model
# These models were trained as described in Chapter 4
try:
    price_model = joblib.load('models/xgboost_pricing_v1.pkl')
    package_model = joblib.load('models/rf_package_v1.pkl')
    print("Models loaded successfully.")
except Exception as e:
    print(f"Error loading models: {e}")

@app.route('/predict_price', methods=['POST'])
def predict():
    data = request.get_json()
    # Feature Extraction
    features = np.array([[
        data['total_items'],
        data['fragile_count'],
        data['distance'],
        data['move_hour']
    ]])

    # 1. Predict Package Type (Classification)
    rec_package = package_model.predict(features)[0]

    # 2. Predict Price (Regression)
    # The package type is encoded and added to features for pricing accuracy
    est_price = price_model.predict(features)[0]

    # Logic for minimum base fare
    final_price = max(1500, round(float(est_price), 2))
    return jsonify({
        'price': final_price,
        'package': rec_package,
        'status': 'success'
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```